

Modeling Divergence Between Community-Perceived and Automatically Calculated MTG Commander Brackets

Henrique Bidarte Massuquetti

Marco Antônio Chitolina da Silva

Abstract

In *Magic: The Gathering* (MTG) Commander, players assign each deck a power-level “bracket” on a five-point ordinal scale, intended as a coordination signal for casual play. Community calculators produce a parallel rule-based estimate of the same quantity from the decklist alone, and the two readings frequently disagree. Our goal is to understand that disagreement: where it lives, how large it is, and which deck-level signals drive it. To do so, we train predictive models on the community label and use them as probes — a model that learns to imitate community judgements lets us isolate which deck features the community implicitly reacts to. We use two complementary representations of the same decks as parallel probes. Bag of Cards (BC), a sparse count vector indexed by card identity, asks whether the community label is driven by specific cards. Deck Features (DF), a dense vector of 102 hand-engineered structural attributes, asks whether it is driven by aggregate deck structure. On a dataset of 12,135 public Archidekt decks with both an author-assigned bracket y_{arch} and an EDHPowerLevel bracket y_{calc} in $\{2, 3, 4\}$, six classifiers per representation are tuned and evaluated under 3-repeat, 5-fold nested cross-validation, using only y_{arch} as the supervised target. DF outperforms BC for every tuned algorithm: the strongest individual model is DF Gradient Boosting at macro-F1 0.6908 ± 0.0093 , and a top-3 hard-voting ensemble reaches 0.6941 ± 0.0096 . When we compare the resulting predictions against the calculator (never used at training time), DF Random Forest aligns with y_{calc} on 69.3% of decks, exceeding the 60.9% agreement between the raw labels. Permutation importance on DF and lift analysis on BC implicate the same signals on both sides — Game Changer count, mass land denial, combo references, tutors, and a small set of recognisable high-power cards. We do not adjudicate between the community and the calculator; we describe where the two readings agree, where they diverge, and what explains the gap.

1 Introduction

Magic: The Gathering (MTG) is a collectible card game in which players build decks from a large card pool and compete under format-specific rules. Commander, the format studied here, is multiplayer and broadly social: each player brings a 100-card singleton deck led by a legendary creature, and a substantial fraction of the experience is determined before the first turn, in a brief discussion of expected deck power. The format provides no matchmaking; it relies on this pre-game negotiation instead.

Commander Brackets are the official five-level vocabulary used for that discussion. Bracket 1 corresponds to thematic, low-power decks; bracket 5 to competitive EDH (cEDH); brackets 2–4 cover the casual middle where the bulk of negotiation occurs. Authors assign brackets to their own decks, combining explicit rule-based constraints with subjective judgement. Several community calculators offer an alternative: they parse the same decklist and produce a rule-based estimate from card identities, lists of cards flagged as high-impact (Game Changers, tutors, combo pieces), prices, and popularity. The two readings frequently disagree.

The central question is empirical: *how far apart are the two readings, and which deck-level signals account for the gap?* The first half is answered by direct counting (Section 3.4). The second is harder, because the community label is subjective and we have no closed-form description of what the community reacts to. To probe it, we train predictive models on the community label and use them as instruments: a classifier that learns to imitate the author’s bracket from observable deck content tells us what the community label is correlated with, and the features the model relies on most heavily are candidates for explaining the gap.

To separate two natural hypotheses about what drives the community label — specific cards versus aggregate deck structure — we deliberately use two representations of the same decks in parallel. Bag of Cards (BC) is a sparse count vector indexed by card identity ($\approx 11\text{k}$ dimensions); DF is a dense vector of 102 hand-engineered structural attributes (mana curve, card-type balance, Game Changer count, tutor count, combo density). Under BC the model can latch onto staples, combo pieces, and high-impact

singletons by name; under DF it sees only aggregated properties of the deck and has no access to card identity. Comparing the two answers a methodologically clean question and feeds directly into the interpretability stage, where the best DF model and the best BC model are inspected separately.

We collected 12,135 public Commander decks from Archidekt for which both y_{arch} and the EDHPowerLevel bracket y_{calc} lie in $\{2, 3, 4\}$. A low-cost spot-checking pass over seven algorithms with distinct inductive biases retains six, which are then tuned per representation under 3-repeat, 5-fold nested cross-validation with per-fold preprocessing and shared splits. y_{calc} is never used at training time; only y_{arch} is the supervised target. The out-of-fold predictions are aligned with y_{calc} post hoc, so that we can ask which models, trained only on the community label, end up closest to the calculator. We additionally verify that completeness checks, GroupKFold by commander identity, and paired statistical tests are consistent with the headline ranking.

Research question. *To what extent do Archidekt-assigned Commander brackets diverge from automatically calculated bracket estimates, and which deck characteristics explain the disagreement?*

Contributions.

1. A curated dataset of 12,135 high-visibility Commander decks with both y_{arch} and y_{calc} labels in $\{2, 3, 4\}$, released along with extraction code.
2. A direct measurement of the raw divergence between y_{arch} and y_{calc} on the modeling set, including signed distribution and structural correlates.
3. A comparison of two deck representations (BC and DF) across six classifiers under repeated nested cross-validation, with per-fold preprocessing and shared splits.
4. A voting analysis over the top 3, 5, and 7 models by macro-F1, plus a separate selection of the best BC and DF models.
5. A comparison between y_{arch} , $\hat{y}_{\text{arch}}$, and y_{calc} , followed by interpretability analyses for the best DF and BC models.

2 Background

This section introduces only the game context required to follow the data and experimental setup. Game-specific terms used downstream (Game Changers, tutors, combos, EDHREC rank, salt score, layouts, keywords) are defined in Appendix A.

2.1 MTG and Commander

MTG is a turn-based collectible card game first published in 1993 [5]. Players build decks from a pool of tens of thousands of cards, draw an initial hand, and cast spells by paying their cost in a resource called *mana*, produced primarily by *land* cards. Each non-land card carries a converted mana cost (CMC) and belongs to one of seven primary types (creature, instant, sorcery, artifact, enchantment, planeswalker, land). MTG defines several formats; we restrict our attention to Commander.

Commander, also known as Elder Dragon Highlander (EDH), is defined by three constraints: each deck contains exactly 100 cards, one of which is the *commander* (a legendary creature playable from a separate zone at any point); the other 99 cards must respect the commander’s *color identity*; and, except for basic lands, each card may appear at most once per deck (the *singleton* rule). A standard game has four players, each starting at 40 life. Because every player brings a different list, a single table can mix widely different strategies, budgets, and expectations. The format provides no matchmaking; balance comes entirely from the pre-game discussion of expected deck power, and that is what the bracket system is designed to support.

2.2 Commander Brackets (community label y_{arch})

The Commander Rules Committee published the five-level bracket system in October 2024 [4]. Bracket 1 corresponds to relaxed, thematic decks; bracket 5 to cEDH, decks tuned to compete with other tuned decks; brackets 2–4 cover the casual middle. We focus on this middle range because it accounts for nearly all of the publicly available data and because the endpoints address qualitatively different use cases.

The bracket rules supply constraints rather than a complete classifier. They cap Game Changer counts and describe broad expectations around speed, consistency, and disruption, but the final assignment remains up to the author — two players may reasonably assign different brackets to nearly identical decks because they weigh consistency, combo density, local norms, and threat level differently. In our data, y_{arch} is the bracket the author selected on Archidekt [1].

2.3 Automated calculators (calculator label y_{calc})

A small number of community tools accept a decklist as input and return a bracket assignment. In this work we use EDHPowerLevel [2], a public web application whose internal rules are not formally documented; we therefore treat it as a black box. From its outputs and accompanying breakdown screens, the tool clearly reacts to cards on curated lists (Game Changers, tutors, extra-turn effects, mass land denial), to known combo pieces, and to card prices and EDHREC popularity [3], but we do not attempt to reverse-engineer the exact aggregation function. What matters for this paper is the input/output behaviour: the calculator observes only the card list (and time-varying external statistics it pulls in), so y_{calc} cannot encode player intent, table norms, or piloting skill, and its output changes when prices or popularity move even if the deck does not.

Disagreement between the two labels is common: a deck the author assigned to bracket 3 may receive bracket 2 or 4 from the calculator. Neither label is treated as ground truth — y_{arch} reflects the author’s reading and y_{calc} reflects one specific rule set — and our objective is to characterise the misalignment, not to adjudicate between the two sources.

3 Dataset Construction

3.1 Data sources

Community labels are obtained from **Archidekt**, a public deck-building platform that stores user-constructed decklists, view counts, and the bracket assigned by each author. Archidekt exposes a paginated REST API, which we used to retrieve raw deck payloads (full mainboard and per-deck metadata).

Calculator labels are obtained from **EDHPowerLevel**, a web application that takes a pasted decklist and returns a bracket assignment alongside a continuous power score. Because EDHPowerLevel exposes no programmatic API, we automated its interface using a headless Chromium browser driven by Playwright: each decklist was pasted into the page and the resulting bracket was read from the rendered DOM. Both extraction passes operated against public endpoints only, with conservative per-request pacing and no credentials.

Collection window and snapshots. Collection took place in May 2026. All time-varying signals — card prices, EDHREC rank, salt score, the curated Game Changers / tutors / mass land denial / atomic combo lists, and y_{calc} itself — are snapshots taken at that point in time, despite the fact that many of the decks under study were created earlier. The resulting temporal mismatch is a known limitation and is discussed in Section 8.1.

3.2 Extraction pipeline

The Archidekt pass returned 14,421 deck records across the brackets we queried. After removing 1,467 duplicate IDs (identical decks appearing across multiple paginated pages) and 4 records whose card-list fingerprints collided, 12,950 unique decks remained. Each was then submitted to EDHPowerLevel to obtain y_{calc} and the auxiliary power score.

The Archidekt query filtered records upstream, so every retained deck already satisfies all of:

- public on Archidekt, format Commander;
- author-assigned bracket $y_{\text{arch}} \in \{2, 3, 4\}$;
- at least 1,000 views;
- exactly 100 cards in the mainboard;
- mainboard only (maybeboard, sideboard, tokens, and extras were ignored).

Joining y_{calc} introduced an additional filter: 815 decks (6.3%) received $y_{\text{calc}} \in \{1, 5\}$ and were excluded from the supervised setup. These are the sparse tails of the bracket scale — bracket 1 is reserved for low-power, thematic decks, and bracket 5 corresponds to cEDH — and are qualitatively distinct from the middle of the spectrum. Retaining them would force the classifier to allocate capacity to a few hundred edge cases rather than to the range where the actual community negotiation takes place. What remains is the **modeling set of 12,135 decks**, with both labels in $\{2, 3, 4\}$.

3.3 Labels

Each deck carries two labels. The community label y_{arch} is the bracket the author assigned on Archidekt; the calculator label y_{calc} is what EDHPowerLevel returned for the same card list. Neither label is treated as ground truth at any point in the analysis. The community label is subjective and varies between authors; the calculator label encodes a single deterministic rule set, fed by time-varying inputs (prices, popularity). All supervised models are trained against y_{arch} exclusively; y_{calc} is joined back after training solely for descriptive comparison.

3.4 Exploratory analysis of the divergence

Before fitting any model, it is informative to characterise how the two labels relate. Table 1 is the descriptive backbone for the dataset, reporting both the joint distribution of y_{arch} and y_{calc} and their marginals. Bracket 3 dominates both label sources, yet the off-diagonal mass is large enough to motivate the rest of this work.

Table 1: Joint counts $y_{\text{arch}} \times y_{\text{calc}}$ on the modeling set.

$y_{\text{arch}} \setminus y_{\text{calc}}$	2	3	4	Total
2	1,014	1,153	154	2,321
3	769	3,909	1,539	6,217
4	128	997	2,472	3,597
Total	1,911	6,059	4,165	12,135

The diagonal of Table 1 contains 7,395 decks, so the two labels match exactly on 60.9% of the modeling set. Allowing a one-bracket tolerance brings agreement to 97.7%; disagreements larger than one bracket occur in only 282 decks (2.3%). The disagreement is also directional. Defining $\Delta = y_{\text{calc}} - y_{\text{arch}}$, the calculator assigns a higher bracket than the author in 23.5% of decks and a lower bracket in 15.6%, with a mean absolute difference of 0.414. When the two readings disagree, the calculator is therefore more likely to push the bracket upward than downward.

Only after this descriptive comparison do we move to the supervised setup. The community label y_{arch} serves as the supervised target throughout; the calculator label y_{calc} is retained as an external reference. To investigate whether deck power is better captured by individual card identities or by aggregate structural summaries, two representations of the same decks are built in parallel. Figure 1 illustrates both: Bag of Cards encodes the decklist as a sparse count vector over the card vocabulary, while Deck Features encodes the same deck as a dense vector of structural attributes.

3.5 Bag of Cards (BC)

Each deck is encoded as a sparse vector indexed by the training-fold card vocabulary, with each dimension storing the count of that card in the deck. Under the singleton rule, most non-basic cards appear at most once; only basic lands can occur in multiples.

The vocabulary is rebuilt inside every fold from the training partition alone. Cards unseen during training are dropped at test time, and any card appearing in fewer than 10 training decks is pruned ($\text{min_df} = 10$). Pruning removes near-zero-variance columns that would otherwise inflate dimensionality without adding usable signal. The cutoff was selected by sweeping $\{5, 10, 20\}$ during spot-checking (Section 4). After pruning, the resulting matrix has approximately 11,000 columns.

Crucially, BC keeps card identity visible to the model. Staples, combo pieces, and high-impact singletons all appear as their own dimensions, so the model is free to associate individual cards with the community bracket.

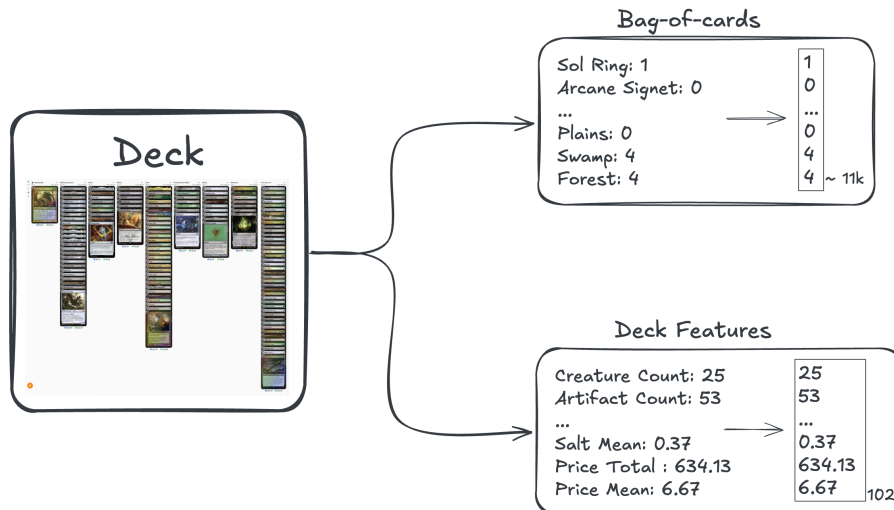


Figure 1: The two representations built from the same deck: Bag of Cards (sparse vector of card counts, $\approx 11k$ dimensions) and Deck Features (dense vector of 102 structural attributes).

3.6 Deck Features (DF)

DF encodes each deck as a dense vector of 105 structural attributes extracted from the Archidekt JSON payload. Features were selected by hand to capture deck construction rather than authoring style. Each attribute belongs to a familiar Commander-relevant category: mana curve, card-type balance, combo density, price, popularity, bracket-specific flags, and so on. Three attributes (`has_companion`, `mainboard_count`, `two_card_combo_singleton_count`) are constant within the modeling subset and are eliminated by the fold-fitted zero-variance filter, leaving 102 features ultimately presented to each classifier. The complete catalog, with the exact computation for each attribute, is provided in Appendix C.

- **Mana base and curve.** Land count, basic / non-basic split, mana production per color, and CMC summary statistics plus per-bucket counts.
- **Card types.** Counts of creatures, instants, sorceries, artifacts, enchantments, planeswalkers, and non-land permanents.
- **Bracket-relevant flags.** Counts of Game Changers, tutors, extra-turn effects, and mass land denial cards — the four signals the bracket system uses to set hard caps.
- **Combo density.** Cards involved in atomic combos, in two-card combos, and total combo references.
- **Popularity and price.** EDHREC rank statistics, salt score statistics, total and per-card deck price.
- **Rarity.** Counts of common, uncommon, rare, and mythic cards.
- **Keywords.** Counts of evergreen keywords, total and distinct keyword counts.
- **Colors and identity.** Color count and per-color presence flags.
- **Multiface layouts.** Counts of transform, modal double-faced, split, and adventure cards.

Features derived from free-text user input are deliberately excluded. The Archidekt deck editor lets the author group cards under *categories*, which appear in the payload as arbitrary user-typed strings rather than a fixed vocabulary: a card that one author tags as *Removal* another may tag as *Interaction*, *Wipe*, or simply *Removal Suite*, and a third may not tag at all. An audit found that 22.8% of decks rely solely on author-defined labels that do not map back to the standard functional vocabulary. Treating those strings as features would predominantly encode authoring style (which tag taxonomy the author happens to prefer) rather than properties of the deck itself, so we drop them.

4 Spot-Checking

Nested cross-validation with hyperparameter search is the most expensive stage of the pipeline. We therefore precede it with a low-cost spot-checking pass, whose sole purpose is to eliminate algorithms that are clearly uncompetitive before the tuning stage begins.

4.1 Algorithm pool

Seven candidates were evaluated, chosen to span distinct inductive biases (Table 2). The pool is intentionally broad — including linear, tree-based, probabilistic, margin-based, and distance-based methods — while remaining computationally feasible on both BC and DF without additional engineering.

Table 2: Candidate algorithms and their inductive biases.

Algorithm	sklearn class	Inductive bias
Decision Tree	<code>DecisionTreeClassifier</code>	Tree partitioning
Random Forest	<code>RandomForestClassifier</code>	Bagging
Gradient Boosting	<code>HistGradientBoostingClassifier</code>	Boosting
Naive Bayes	<code>MultinomialNB</code> (BC) / <code>GaussianNB</code> (DF)	Probabilistic
Logistic Regression	<code>LogisticRegression</code>	Linear
Linear SVC	<code>LinearSVC</code>	Linear
K-Nearest Neighbors	<code>KNeighborsClassifier</code>	Distance-based

4.2 Hold-out protocol

Each candidate is run on both representations with scikit-learn 1.5 defaults (Appendix D lists them explicitly). We use 5 stratified 80/20 hold-out splits with seeds $\{1, 2, 3, 4, 5\}$, stratified by y_{arch} , and report the mean and standard deviation of macro-F1 across seeds. Spot-checking is deliberately not a tuning pass: leaving hyperparameters fixed at their defaults ensures that the resulting ranking reflects the algorithm itself, not the amount of effort invested in its grid.

4.3 Results and selection

Table 3 reports mean macro-F1 per (algorithm, representation) pair together with the per-representation rank. The two rankings agree at the bottom: KNN places last on both sides and shows the largest variance, so we drop it. Gradient Boosting, Logistic Regression, Linear SVC, Decision Tree, and Naive Bayes all appear in the top five for both representations. Random Forest is the only asymmetric case — rank 2 on DF, rank 6 on BC.

Table 3: Spot-checking macro-F1 (mean over 5 hold-out seeds) per algorithm and representation, with per-representation ranks. ✓ marks membership in A_{union} used by nested CV.

Algorithm	DF mean	DF rank	BC mean	BC rank	In A_{union}
Gradient Boosting	0.6850	1	0.6342	1	✓
Random Forest	0.6686	2	0.5243	6	✓
Logistic Regression	0.6672	3	0.5927	2	✓
Linear SVC	0.6225	4	0.5459	4	✓
Decision Tree	0.6042	5	0.5445	5	✓
Naive Bayes	0.5792	6	0.5570	3	✓
KNN	0.5241	7	0.4541	7	—

Let A_{BC} and A_{DF} denote the top-five algorithms on BC and DF respectively, and let $A_{\text{union}} = A_{\text{BC}} \cup A_{\text{DF}}$. We carry A_{union} forward into nested CV rather than picking one top-five list because the goal at this stage is to triage *algorithms*, not specific (algorithm, representation) pairs — the BC-vs-DF asymmetry of Random Forest, for instance, may close once hyperparameters are tuned, and we would rather keep an algorithm we may regret dropping. Concretely, the union here is $\{\text{Decision Tree, Random Forest, Gradient Boosting, Naive Bayes, Logistic Regression, Linear SVC}\}$: six algorithms, each evaluated

on both representations, for a total training surface of $6 \times 2 = 12$ tuned models. The full table with standard deviations and the `bc_min_df` sweep that fixed `min_df = 10` are provided in Appendix D.

5 Training and Model Selection

5.1 Nested cross-validation

Nested cross-validation is used to keep hyperparameter selection separated from the final estimate of generalization performance:

- **Outer loop.** `StratifiedKfold` with 5 splits, repeated with 3 independent seeds (1, 2, 3), yielding 15 outer evaluations per model.
- **Inner loop.** `StratifiedKfold` with 3 splits (`seed = outer_seed + 100`). The inner loop performs the grid search, selecting the best configuration for each outer fold without ever observing the corresponding outer test set.

All 12 models share the same outer and inner fold indices. Indices are derived from the label vector and the seeds at runtime, so any machine with the same dataset and dependency versions reproduces the same splits. Because every model is evaluated on the same held-out rows, paired statistical tests across models are well-defined.

5.2 Per-fold preprocessing

All preprocessing is fit on the training partition of the current fold and applied to the held-out partition without ever observing it. No statistic is computed on the full dataset prior to splitting, so test-fold information cannot leak into the model through imputation values, outlier caps, or scaling parameters. The full rationale (median imputation, winsorization of price, and the selective application of scaling) is provided in Appendix B; in summary:

- **Imputation.** Missing EDHREC ranks and salt scores are filled with the training-fold median.
- **Winsorization.** `price_total` is clipped at the training-fold 99th percentile; the long tail (a handful of decks above \$50,000) would otherwise dominate linear models and waste tree splits.
- **Scaling.** `StandardScaler` is applied to DF features for the scale-sensitive algorithms (Logistic Regression, Linear SVC, KNN). Tree-based models and Gaussian Naive Bayes receive unscaled DF features.
- **Leakage prevention.** The model sees only the current training partition. All fields derived from `EDHPowerLevel` (`ycalc`, the continuous power score, and any column prefixed `ep1_`) are stripped from every feature matrix, in every fold. The only training target is `yarch`; `ycalc` is joined back after training, only for descriptive comparison.

5.3 Hyperparameter grids

Each algorithm uses a 24-configuration grid covering the most impactful hyperparameters without inflating the search cost (Table 4). Inner-loop selection is performed by macro-F1.

5.4 Evaluation metrics

The primary metric is macro-F1, which averages per-class F1 with equal weight across classes. This choice is important here because bracket 3 accounts for 51.2% of the modeling set, and plain accuracy would obscure poor recall on the minority classes. We additionally report accuracy, macro-precision, macro-recall, and per-class confusion matrices, all as mean $\pm$ standard deviation over the 15 outer folds.

Table 4: Hyperparameter grids for nested CV (24 configurations per algorithm).

Algorithm	Parameter	Values
Decision Tree	<code>max_depth</code>	{None, 10, 20}
	<code>min_samples_leaf</code>	{1, 5}
	<code>ccp_alpha</code>	{0, 0.005}
	<code>class_weight</code>	{None, balanced}
Random Forest	<code>n_estimators</code>	{100, 250, 500}
	<code>max_features</code>	{sqrt, log2}
	<code>min_samples_leaf</code>	{1, 2}
	<code>class_weight</code>	{None, balanced}
Gradient Boosting	<code>max_iter</code>	{200, 500}
	<code>learning_rate</code>	{0.05, 0.1, 0.2}
	<code>max_leaf_nodes</code>	{15, 31}
	<code>class_weight</code>	{None, balanced}
Naive Bayes (BC)	<code>alpha</code>	12 log-spaced values in $[10^{-3}, 100]$
	<code>fit_prior</code>	{True, False}
Naive Bayes (DF)	<code>var_smoothing</code>	24 log-spaced values in $[10^{-12}, 10^{-3}]$
Logistic Regression	<code>C</code>	{0.001, 0.1, 1, 100}
	<code>class_weight</code>	{None, balanced}
	<code>l1_ratio</code>	{0, 0.5, 1} (ElasticNet penalty)
Linear SVC	<code>C</code>	{0.001, 0.01, 0.1, 1, 10, 100}
	<code>class_weight</code>	{None, balanced}
	<code>penalty</code>	{L1, L2}

5.5 Results

All 12 models completed the full 15 outer folds. Table 5 reports the nested-CV results side by side for DF and BC, with per-representation ranks. The strongest individual model is DF Gradient Boosting, with macro-F1 0.6908 ± 0.0093 ; the strongest BC model is BC Gradient Boosting, with macro-F1 0.6433 ± 0.0121 . DF outperforms BC for every algorithm in the tuned comparison: the DF-BC gap in macro-F1 ranges from +0.0370 for Naive Bayes to +0.1251 for Decision Tree. On this dataset, the dense handpicked representation yields a more useful signal than sparse card identity alone for every algorithm we tried.

Table 5: Nested-CV results for DF and BC models side by side. Macro-F1 is mean $\pm$ std over 15 outer folds; ranks are within each representation. Bracket-relevant flags and curated lists in DF (Game Changers, tutors, mass land denial, combos) overlap with the calculator’s inputs; this is discussed as a circularity caveat in Section 8.1.

Algorithm	DF			BC		
	Macro-F1	Acc.	Rank	Macro-F1	Acc.	Rank
Gradient Boosting	0.6908 ± 0.0093	0.6978	1	0.6433 ± 0.0121	0.6442	1
Random Forest	0.6733 ± 0.0076	0.7034	2	0.6326 ± 0.0122	0.6534	2
Decision Tree	0.6727 ± 0.0100	0.6876	3	0.5475 ± 0.0116	0.5540	6
Logistic Regression	0.6707 ± 0.0110	0.6944	4	0.6236 ± 0.0096	0.6212	3
Linear SVC	0.6557 ± 0.0085	0.6584	5	0.6147 ± 0.0118	0.6281	4
Naive Bayes	0.5905 ± 0.0093	0.5874	6	0.5535 ± 0.0090	0.5636	5

The best Gradient Boosting model converged to the same modal configuration in both representations across all 15 outer folds: balanced class weights, learning rate 0.05, 200 boosting iterations, and 31 leaf nodes. Full per-fold hyperparameters are in Appendix E.

To check that the ranking in Table 5 is not just sampling noise across folds, we run paired tests on the 15 shared outer scores. A Friedman test over the 12 models rejects the hypothesis of equal ranks (statistic = 158.84, $p < 10^{-6}$), confirming that not all models perform equally. The post-hoc Nemenyi test, with critical difference 4.3025 ranks at $\alpha = 0.05$, separates DF Gradient Boosting (mean rank 1.07) from every BC model with statistical significance; among the top DF models, pairwise Wilcoxon tests return non-significant differences only between DF Decision Tree, DF Random Forest, and DF Logistic

Regression, which sit within 0.003 macro-F1 of each other.

A second concern is whether the models are simply memorising commander-specific patterns: since the commander is included in every deck, a model could in principle learn “deck-with-Atraxa $\rightarrow$ bracket 4” rather than anything generalisable. To check this, we ran a 5-fold `GroupKFold` grouped by commander identity, with the group key set to the *sorted* set of commander `oracle_uids` so that partner pairs and partner-with-background pairs are not split across folds. The top models barely move under this stricter split — DF Gradient Boosting scores 0.7021 (versus 0.6908 stratified) and BC Gradient Boosting 0.6496 (versus 0.6433) — so the leading models are not relying on commander identity to generalise.

5.6 Voting ensembles

Three simple hard-voting ensembles are constructed directly from the saved out-of-fold predictions: the top 3, top 5, and top 7 individual models ranked globally by mean macro-F1. No base model is retrained. Ties are resolved by the summed macro-F1 of the models voting for each class, with the smaller bracket as a deterministic fallback when a tie persists. Table 6 reports the macro-F1 of each ensemble together with its size; the full member lists are in Appendix F. Each ensemble strictly extends the previous one ($\text{top-3} \subset \text{top-5} \subset \text{top-7}$).

Table 6: Hard-voting ensembles built from OOF predictions. Δ vs best is the macro-F1 gain over the strongest individual model (DF Gradient Boosting at 0.6908).

Ensemble	Size	Macro-F1	Δ vs best
<code>voting_top3</code>	3	0.6941 ± 0.0096	+0.0033
<code>voting_top5</code>	5	0.6932 ± 0.0076	+0.0024
<code>voting_top7</code>	7	0.6936 ± 0.0085	+0.0028

The best ensemble, `voting_top3`, improves over the strongest individual model by only +0.0033 macro-F1. The gain is measurable yet small, suggesting that the top DF models already capture most of the learnable signal in y_{arch} . Adding the fourth and fifth DF models, and subsequently the two strongest BC models, does not surpass the top-3 vote.

5.7 Selected models

After training, three selections are carried forward (Table 7). The best ensemble overall is `voting_top3`; the best DF model and best BC model are both Gradient Boosting in their respective representations. DF Gradient Boosting is also the best individual model overall, which we mark with an asterisk in the table. Interpretability is performed on the two per-representation models rather than on the ensemble, since the ensemble combines several mechanisms and does not expose a single coherent feature space.

Table 7: Model selections used after training. The asterisk marks DF Gradient Boosting as the best individual model overall.

Role	Model	Repr.	Macro-F1
Best DF	<code>df_gradient_boosting*</code>	DF	0.6908 ± 0.0093
Best BC	<code>bc_gradient_boosting</code>	BC	0.6433 ± 0.0121
Best ensemble	<code>voting_top3</code>	DF	0.6941 ± 0.0096

6 Model-Calculator Comparison

This section is purely descriptive: no model is trained or retrained, and y_{calc} is never used as a target. The analysis reads the out-of-fold predictions saved by nested CV and voting, and compares them to the calculator label on the same deck rows.

Since the outer evaluation uses 3 repeats over the 12,135-deck modeling set, the OOF comparison contains 36,405 rows. Each deck has a stable y_{calc} but contributes one prediction per repeat; for this reason, the OOF confusion matrices in the appendix sum to 36,405 rather than 12,135.

Two baselines are used. The first is the direct label comparison y_{arch} versus y_{calc} , already shown in Table 1. The second is the model comparison $\hat{y}_{\text{arch}}$ versus y_{calc} . We report exact agreement, within-one-bracket agreement, mean absolute difference, and macro-F1 against y_{calc} . Exact agreement answers the question of how often each source matches the calculator; macro-F1 against y_{calc} answers the same question while accounting for class imbalance. The gap column has a different interpretation: it is defined as $|\text{macro-F1}(y_{\text{arch}}) - \text{exact agreement}(\hat{y}_{\text{arch}}, y_{\text{calc}})|$, and quantifies how far a model’s performance on the Archidekt target sits from its raw agreement with the calculator. A large gap indicates that a model is a strong predictor of y_{arch} without being the one closest to y_{calc} .

Table 8: Calculator-alignment highlights. The raw-label row compares y_{arch} directly with y_{calc} ; all other rows compare a model’s out-of-fold predictions $\hat{y}_{\text{arch}}$ with y_{calc} . In the Source column, GB is Gradient Boosting, RF is Random Forest, DT is Decision Tree, and NB is Naive Bayes. Columns: *Exact* — fraction of rows where $\hat{y}_{\text{arch}} = y_{\text{calc}}$; ± 1 — fraction within one bracket; *Mean $|\Delta|$* — mean absolute difference; *F1 calc* — macro-F1 of the predictions when scored against y_{calc} ; *F1 arch* — macro-F1 against y_{arch} (the supervised target, taken from Table 5); *Gap* — $|F1 \text{ arch} - \text{Exact}|$, measuring how far performance on the community target sits from raw calculator agreement.

Case	Source	Exact	± 1	Mean $ \Delta $	F1 calc	F1 arch	Gap
Raw labels	y_{arch}	60.9%	97.7%	0.414	0.5843	—	—
Max agreement	DF RF	69.3%	99.5%	0.312	0.6674	0.6733	0.0194
Min agreement	BC DT	54.2%	96.6%	0.492	0.5328	0.5475	0.0057
Max gap	DF GB	64.0%	98.1%	0.379	0.6323	0.6908	0.0508
Min gap	BC NB	55.5%	97.1%	0.474	0.5462	0.5535	0.0010
Best BC	BC GB	60.3%	97.3%	0.424	0.5984	0.6433	0.0402
Best ensemble	Top-3 vote	66.7%	98.7%	0.346	0.6499	0.6941	0.0276

The table is dense; we read it through four questions.

Q1. Which representation gets closer to the community? DF, by a wide margin. Table 5 shows DF outperforming BC on y_{arch} for every algorithm; the strongest DF model beats the strongest BC model by +0.0475 macro-F1. The community label is more about *what kind of deck this is* than about *which specific cards are in it*.

Q2. Which representation gets closer to the calculator? Also DF. Every model that exceeds the raw 60.9% baseline in the Exact column of Table 8 is a DF model (DF Random Forest at 69.3%, top-3 vote at 66.7%, DF Gradient Boosting at 64.0%); BC Gradient Boosting nearly matches the raw labels (60.3%) and other BC models sit below. This is partially circular — DF counts the same kinds of cards EDHPowerLevel counts (Section 8.1) — but informative: DF has no feature for which specific Game Changer is present, so the alignment is driven by aggregates, not identity.

Q3. Which algorithm aligns most with the community vs the calculator? Not the same one. The best community predictor is DF Gradient Boosting (macro-F1 0.6908 on y_{arch} , but only 64.0% agreement with the calculator — the largest gap in the table). The best calculator-aligned model is DF Random Forest (69.3% agreement, macro-F1 0.6733 on y_{arch} — 0.0175 below the best). Gradient Boosting is willing to over- or under-shoot the calculator when that improves community-label accuracy; Random Forest’s bagged majority vote lands closer to the calculator’s rule-driven behaviour even though both were trained on y_{arch} .

Q4. What does the gap mean? A high gap (DF Gradient Boosting, 0.0508) marks a model that captures community-specific structure not aligned with the calculator’s rules. A low gap (BC Naive Bayes, 0.0010) marks a weak classifier roughly equally bad at both targets — not one that has learned the calculator. Closeness to the calculator is informative only conditional on first being good at y_{arch} . Together: the community label and the calculator react to overlapping but not identical structure; deck-level aggregates explain most of the overlap, and the residual — where DF Gradient Boosting’s gap lives — is what the community sees and the calculator does not.

7 Interpretability

Interpretability is performed on exactly the best individual model per representation, as selected in Section 5.7: DF Gradient Boosting and BC Gradient Boosting. The per-representation split is deliberate: the best global model is a DF model and the best ensemble combines several estimators, yet the paper needs to characterise both representations. We therefore consider two representation-specific questions. For DF, which handpicked deck attributes does the best structured model rely on? For BC, which card identities are enriched in each predicted bracket?

The reported performance estimates remain those from nested CV. For interpretation only, each selected model is fit again on the full modeling set or on a dedicated interpretability split, depending on the method. These refits are not used to claim new predictive performance.

7.1 Deck Features

For DF, we compute permutation importance on a stratified 20% hold-out with 10 repeats. HistGradient-Boosting does not expose impurity-based feature importances, and permutation importance measures the drop in macro-F1 after shuffling a feature column. A large value indicates that the model loses predictive performance when that feature is decoupled from the label. Table 9 reports the top features.

Table 9: Top DF permutation importances for `df_gradient_boosting`.

Feature	PI mean	PI std
<code>game_changer_count</code>	0.2281	0.0096
<code>mass_land_denial_count</code>	0.0288	0.0029
<code>unique_atomic_combo_refs_count</code>	0.0143	0.0052
<code>tutor_count</code>	0.0097	0.0038
<code>cmc_1_count</code>	0.0060	0.0010
<code>land_count</code>	0.0043	0.0019

The ranking is highly concentrated. The feature `game_changer_count` attains permutation importance 0.2281, nearly an order of magnitude above the second-ranked `mass_land_denial_count` at 0.0288. The model does not depend on a single feature, but the sharpest split between brackets is tied to a signal that the bracket system explicitly relies on. The direction is consistent with the bracket definition: on the validation split, mean `game_changer_count` rises from 0.001 in decks predicted as bracket 2 to 1.499 in bracket 3 and 4.146 in bracket 4. Subsequent features refine the same notion of power — mass land denial captures resource denial, combo references capture deterministic wins, and tutor count captures consistency.

The same pattern carries over into the calculator comparison. OOF rows where $\hat{y}_{\text{arch}} = y_{\text{calc}}$ carry higher `game_changer_count` than divergent rows (2.165 versus 0.939). When clear bracket-defining signals are present, the model and calculator tend to agree; divergence is concentrated in decks whose power level depends on context, synergy, or author interpretation rather than on the explicit bracket flags.

7.2 Bag of Cards

For BC, dense permutation importance over more than 11k card features would require repeatedly shuffling a very large sparse matrix and is computationally impractical. We instead analyse card lift over OOF predictions:

$$\text{lift}(k, c) = \frac{P(c \text{ present} \mid \hat{y}_{\text{arch}} = k)}{P(c \text{ present})}.$$

Cards present in fewer than 10 unique decks are filtered. Intuitively, the lift measures how much more common a card is inside a predicted bracket than across the dataset as a whole; a value of 3 indicates that the card appears approximately three times as often in decks predicted as that bracket. Table 10 reports the strongest bracket 4 lifts.

The bracket 4 list is dominated by cards associated with high-power Commander play: fast mana, tutors, and resource denial. Several of the top cards are not merely efficient but actively constrain the actions available to the table. *Blood Moon*, *Winter Moon*, *Ruin*, and *Back to Basics* attack mana bases; *Chrome Mox* and *Grim Monolith* accelerate early turns; *Imperial Seal* increases consistency by

Table 10: Top bracket 4 card lifts for `bc_gradient_boosting`.

Card	Lift	Freq. in predicted bracket 4
<i>Blood Moon</i>	3.325	3.5%
<i>Winter Moon</i>	3.325	2.1%
<i>Ruination</i>	3.325	0.5%
<i>Harbinger of the Seas</i>	3.278	1.3%
<i>Back to Basics</i>	3.186	0.8%
<i>Chrome Mox</i>	3.185	6.7%
<i>Imperial Seal</i>	3.160	2.1%
<i>Grim Monolith</i>	3.138	2.1%

tutoring for a specific card. This is the card-identity counterpart of the signal that DF surfaces: high brackets correlate with faster, more consistent, and more restrictive deck construction.

A separate lift analysis distinguishes concordant from divergent OOF rows. Concordant rows are enriched for *Blood Moon*, *Imperial Seal*, *Force of Will*, and *Chrome Mox* — cards on which model and calculator broadly agree on high power. Rows in which the model predicts below the calculator ($\hat{y}_{\text{arch}} < y_{\text{calc}}$) are enriched for tempo-altering cards: *Vorinclex*, *Voice of Hunger*, *Expropriate*, *Time Stretch*, and *Time Sieve*, which either grant additional turns or severely restrict opponents’ resources. The calculator counts these as explicit high-impact signals; authors often do not, because they evaluate a deck against a specific playgroup rather than against the bare rules text. The two sources react to overlapping signals but encode different notions of context.

8 Conclusion

This paper has set up a reproducible pipeline for studying the divergence between community-assigned and automatically calculated Commander brackets. On 12,135 Archidekt decks with both labels in brackets 2–4, raw agreement between y_{arch} and y_{calc} is 60.9%, and the calculator assigns higher brackets more often than authors do when the two disagree. Across six algorithms and two representations, DF outperforms BC under nested cross-validation; the strongest individual model is DF Gradient Boosting at macro-F1 0.6908 ± 0.0093 , and the best voting ensemble improves this only marginally to 0.6941.

Interpretability supports a coherent reading of the results. The DF model is dominated by aggregate structure — Game Changer count, mass land denial, combo references, and tutors — while BC lift surfaces the cards that a Commander player would already recognise as bracket-4 indicators: *Blood Moon*, *Chrome Mox*, *Imperial Seal*, and *Mana Vault*. The models do not adjudicate between the community and the calculator; they expose where the two readings overlap and where they diverge.

8.1 Discussion, Limitations, and Future Work

The headline empirical findings are: structured deck features (DF) predict the community bracket better than raw card identity (BC) for every tuned algorithm; the model best aligned with the calculator (DF Random Forest, 69.3%) is not the model best at predicting the community label (DF Gradient Boosting, macro-F1 0.6908); and the same set of deck-level signals — Game Changers, mass land denial, combos, tutors, and a handful of recognisable high-power cards — shows up in both interpretability views. We did not treat either label as ground truth; the contribution is descriptive.

Limitations. The evidence is bounded in several ways.

One community source, one calculator. The community label comes from Archidekt alone and y_{calc} from EDHPowerLevel alone. A different platform’s users may follow different self-assessment conventions, and other calculators apply different rule sets, so the specific alignment numbers in Table 8 would shift if rerun against a different tool. The directional finding — DF aggregates align more strongly than BC card identities — is likely robust, but the 69.3% is not.

The two representations are evaluated separately. BC and DF are deliberately treated as parallel probes and never concatenated. A model that sees both card identity and aggregate structure jointly could reasonably outperform either, but that would conflate the two probes we want to compare.

Sampling frame. The Archidekt query retains only decks with at least 1,000 views, biasing the sample toward polished, public-facing lists. Lowering the threshold to e.g. 500 would enlarge but also dilute it,

which is its own bias rather than a strict improvement.

Shared feature space with the calculator. The DF representation explicitly counts signals that the bracket rules and EDHPowerLevel also count: Game Changers, tutors, mass land denial, extra-turn effects, combo references. Even though y_{calc} is never used at training time, part of the DF-vs-calculator alignment in Section 6 is structurally expected. We flag this as a circularity caveat rather than a corrected number.

Ordinal brackets, unordered loss. Brackets are ordinal ($2 < 3 < 4$), but our classifiers treat them as unordered classes; macro-F1 weights a $2 \rightarrow 4$ confusion the same as a $2 \rightarrow 3$ one. The within-one-bracket agreement we report partially compensates.

Temporal snapshot. y_{calc} and several DF features (prices, EDHREC rank, salt, curated lists) are May 2026 snapshots; decks were built over a longer span. Random CV splits do not control for this mismatch, and a re-query audit on 90 decks showed 3 calculator-label flips after several weeks.

Data redistribution. We re-share only deck IDs, both bracket labels, and derived features — not raw upstream payloads. Long-term reproducibility against the live sites is not guaranteed; we distribute a frozen snapshot together with the extraction code.

Bracket prediction is not win-rate prediction. The models learn labels attached to decklists, not the strength of those decks in actual play.

Future work. The most informative next experiments are local extensions, not new models. (i) A circularity ablation that drops the explicit bracket-rule columns from DF and re-runs the calculator-alignment analysis, separating “DF agrees because it counts the same things” from “DF agrees because the community uses the same signals”. (ii) A second calculator and a second community source (Commander Salt, Moxfield) to test how much of the gap is platform-specific. (iii) Ordinal-aware modelling and metrics (ordinal logistic regression, MAE on bracket values, quadratic-weighted kappa). (iv) A BC+DF model that uses card identity and aggregate structure jointly. (v) A separate experimental design for brackets 1 and 5.

References

- [1] Archidekt – MTG deck builder. <https://archidekt.com>. Accessed: 2026-05-23.
- [2] EDHPowerLevel – commander power level calculator. <https://www.edhpowerlevel.com>. Accessed: 2026-05-23.
- [3] EDHREC – commander recommendations and statistics. <https://edhrec.com>. Accessed: 2026-05-23.
- [4] Commander Rules Committee. Commander Brackets. <https://mtgcommander.net/index.php/2024/10/17/october-17-2024-announcement/>, 2024. Accessed: 2026-05-23.
- [5] Wizards of the Coast. Magic: The Gathering. <https://magic.wizards.com>, 1993. Accessed: 2026-05-23.

The appendices below collect supplementary material that did not need to live in the main text: MTG and Commander background (Appendix A), preprocessing rationale (Appendix B), the full Deck Features catalog (Appendix C), spot-checking detail (Appendix D), training verification and statistical tests (Appendix E), and voting and model-selection detail (Appendix F).

A MTG Terminology and Gameplay

This appendix pulls together the MTG concepts that appear in the body. Readers already familiar with the game can skip ahead; the main text is self-contained on methodology and results.

A.1 Game structure

Commander and command zone. The *commander* is a *legendary creature* that begins the game in a separate zone, the *command zone*, and may be cast from there at any point during the game. The commander’s *color identity* — the set of mana colors appearing in its cost and rules text — constrains which cards the other 99 cards of the deck may include.

Singleton format. Commander is a *singleton* format: each deck may contain at most one copy of any non-basic land card. *Basic lands* — generic resource-producing cards with no unique rules text — may be included in any quantity.

Player count. Commander is typically played in pods of four, though the format admits any number of players from two upward. The four-player pod is the configuration around which the bracket discourse is framed.

A.2 Cards, mana, and the curve

Mana and mana cost. *Mana* is the resource in MTG, produced mostly by *land* cards. Players play at most one land per turn and spend mana to pay the *mana cost* of spells. The *converted mana cost* (CMC), also called *mana value*, is the total amount of mana required to cast a card, regardless of color.

Mana curve. The distribution of CMC values across a deck is the *mana curve*. Cards with CMC 0–1 are cheap and fast; cards with CMC 5+ are expensive and slow. The shape of the curve reflects the deck’s expected speed and consistency.

Card types. MTG defines several primary *card types*: *creatures* attack and block; *instants* and *sorceries* are one-time spells; *artifacts* are objects with persistent effects; *enchantments* are ongoing magical effects; *planeswalkers* are recurring-ability allies; *lands* produce mana. Cards that remain on the battlefield after being played are collectively called *permanents*.

Rarity tiers. MTG cards are printed at four rarity levels that broadly correlate with power and availability: *common* (most frequent), *uncommon*, *rare*, and *mythic rare* (least frequent). A deck’s rarity distribution is a rough proxy for power ceiling and price.

Multiface layouts. Some MTG cards have multiple faces. *Transform* cards flip between two faces under specific conditions. *Modal double-faced cards* let the player choose which face to play at cast time. *Split cards* contain two independent spells on one card. *Adventure* cards bundle a creature with an instant or sorcery.

Mana colors. MTG uses five colors of mana: *White* (W), *Blue* (U), *Black* (B), *Red* (R), and *Green* (G), each associated with a distinct strategic identity.

A.3 Power indicators

Staples. A *staple* is an informal term for a card so generically efficient that it appears in a large fraction of competitive or semi-competitive decks regardless of strategy. Examples: *Sol Ring* (a cheap mana-producing artifact), *Command Tower* (a land that produces any color in the commander’s identity).

Game Changers. *Game Changers* are cards officially designated by the Commander Rules Committee as capable of ending games or dominating tables on their own. Examples: *Thassa’s Oracle*, *Rhystic Study*. Their count in a deck is the primary signal the bracket system uses to distinguish brackets 3 and 4.

Tutors. A *tutor* is a card that searches the player’s library for another specific card and places it in the player’s hand or in play, raising deck consistency. Examples: *Demonic Tutor*, *Enlightened Tutor*.

Combos. A *combo* is a combination of two or more cards that, once assembled, produces a game-winning or overwhelming effect (infinite damage, infinite mana, deck-out). An *atomic combo* is one in which every piece is necessary and sufficient; a *two-card combo* is a particularly efficient variant requiring only two pieces.

Extra-turn effects. Cards that grant a player additional turns beyond the standard cycle (*Time Warp*, *Temporal Manipulation*) create significant tempo advantages and are considered high-impact.

Mass land denial. Cards that destroy or otherwise neutralize all players' lands simultaneously (*Armageddon*, *Ravages of War*) are socially contentious in Commander because they can render the game unplayable for the rest of the table.

A.4 Keywords

Keywords are shorthand labels for recurring rules text. *Evergreen* keywords appear across many sets and are always available; examples include *Flying* (can be blocked only by creatures with Flying or Reach), *Trample* (excess combat damage carries over to the defending player), *Haste* (can attack the turn it enters play), *Hexproof* (cannot be targeted by opponents), and *Deathtouch* (destroys any creature it damages).

A.5 Community resources

EDHREC rank. EDHREC (<https://edhrec.com>) aggregates published Commander decklists and computes per-card popularity statistics. The *EDHREC rank* orders cards by inclusion frequency: rank 1 is the most widely played card, with higher numbers denoting less popular cards.

Salt score. The *salt score* is a community-sourced metric compiled by EDHREC that measures how much frustration a card is reported to cause opposing players. High-salt cards typically lock opponents out, induce repeated random effects, or end games in ways perceived as unfun.

B Preprocessing — Extended Rationale

Section 5.2 lists the four transformations used inside nested CV. This appendix is the why behind each one.

Imputation. EDHREC rank and salt score come from an external community service, and they are missing for cards that EDHREC has not yet indexed (mostly rare or newly released ones). The missing rate is small. We fill those values with the training-fold median because the missingness is a property of the card, not the deck. The median is a better default than the mean because both fields have heavy-tailed distributions (a few cards sit at very high ranks or salt scores); it is also better than zero because zero is not neutral on either scale (EDHREC rank 0 would be the single most popular card, salt 0 a card nobody finds frustrating).

Winsorization. Total deck price (`price_total`) is clipped at the training-fold 99th percentile. Most decks cost a few hundred dollars, but a small collector-grade tail breaks past \$50,000 thanks to a handful of reserved-list cards. Without clipping, scale-sensitive linear models would over-weight that tail, and tree splits would lock onto degenerate thresholds set by a single deck. Winsorization keeps the useful signal and trims the leverage. We picked it over a log transform because the unit stays “dollars,” which makes coefficients and feature importances easier to read downstream.

Scaling. `StandardScaler` centers each DF feature to zero mean and unit variance, fit on the training fold. It is applied to Logistic Regression, Linear SVC, and KNN, where objective functions or distance metrics are scale-dependent and would otherwise be dominated by features like `price_total` or `edhrec_rank_mean`. Tree-based models (Decision Tree, Random Forest, Gradient Boosting) and Gaussian Naive Bayes are invariant to monotonic per-feature rescaling, so they get unscaled features and we avoid a transform that would only blur feature-importance interpretation.

Leakage prevention. The supervised target is y_{arch} . Every column the EDHPowerLevel pass produced (y_{calc} , the continuous power score, efficiency, tipping point, anything prefixed `ep1_`) is stripped from the feature matrix before fitting. The calculator label is only re-joined for the descriptive comparison after training. Without this rule, a trivial classifier that just copied the EDHPowerLevel bracket would score well while explaining nothing about the Archidekt label.

C Deck Features (DF) Catalog

This appendix walks through every feature in the Deck Features representation from Section 3.6, grouped into the same families. Each one is computed strictly from data already in the Archidekt raw payload (mainboard rows, oracle card metadata, printing-level rarity and price); nothing external is queried at training or inference time. Per-card quantities are respected everywhere: a feature defined as a count multiplies the per-card boolean by the row's `quantity`, which matters for basic lands and the handful of cards allowed in multiples.

C.1 Basic structure

- `unique_card_count`: number of distinct `oracle_uids` in the mainboard.
- `non_commander_card_count`: total quantity of mainboard cards that are not the commander.
- `commander_card_count`: total quantity of commander cards (1 for single-commander decks, 2 for Partner pairs and Background combinations).
- `mainboard_count`: total quantity of mainboard cards. Always 100 in the modeling set (the Commander deck-size rule) and removed by the zero-variance filter; included in the feature payload as a structural sanity check.
- `has_companion`: boolean flag indicating whether a Companion card is registered for the deck. Always false in the modeling set because Companions are excluded upstream; removed by the zero-variance filter.
- `view_count`: cumulative number of Archidekt page views for the deck snapshot, used as a coarse popularity proxy.

C.2 Colors and color identity

- `deck_color_count`: cardinality of the union of `colorIdentity` across all mainboard cards, in $\{0, \dots, 5\}$.
- `has_W`, `has_U`, `has_B`, `has_R`, `has_G`: boolean flags, one per color, true if any card in the mainboard has that color in its identity.

C.3 Mana base

- `land_count`, `nonland_count`: total quantities of lands and non-lands in the mainboard.
- `basic_land_count`, `nonbasic_land_count`: split of the land count between basic lands (the five color basics plus Wastes, or any card with `superType=Basic` and `type=Land`) and non-basics.
- `land_mana_production_W`, `_U`, `_B`, `_R`, `_G`, `_C`: for each color (including colorless), the count of mainboard lands whose `manaProduction` entry includes that color.
- `lands_that_produce_multiple_colors_count`: count of lands that produce more than one colored mana (colorless not counted).
- `lands_that_produce_colorless_count`: count of lands whose `manaProduction` includes the colorless “C” symbol.

C.4 Mana curve (CMC)

CMC is taken from the canonical `oracleCard.cmc` field. Archidekt's user-overridable `custom_cmc` is deliberately ignored: an audit showed that about 19% of decks use it on at least one row, mixing legitimate cases (X-spells) with subjective preference, so using it would inject author-specific noise into a feature that is meant to describe the card.

- `cmc_mean`, `cmc_median`, `cmc_std`: mean, median, and population standard deviation of CMC across all mainboard cards (lands counted at CMC 0).

- `nonland_cmc_mean`, `nonland_cmc_median`, `nonland_cmc_std`: same statistics restricted to non-land cards.
- `cmc_0_count`, `cmc_1_count`, ..., `cmc_5_count`, `cmc_6_plus_count`: histogram of non-land cards by integer CMC bucket, with CMCs of 6 or more collapsed into a single bucket.

C.5 Card types

- `creature_count`, `instant_count`, `sorcery_count`, `artifact_count`, `enchantment_count`, `planeswalker_count`: count of mainboard cards (respecting quantity) whose `types` array contains the corresponding type.
- `noncreature_spell_count`: count of mainboard cards that are neither creatures nor lands.
- `nonland_permanent_count`: count of mainboard cards that are non-land permanents (creature, artifact, enchantment, or planeswalker).

C.6 Bracket-relevant flags

These count cards flagged by external sources that the bracket system uses to set hard caps (Appendix A.3). The four boolean flags come directly from the `oracleCard` payload Archidekt returned at collection time (May 2026); they are not recomputed from Rules Committee documents and not refreshed across folds, so they are a single dataset-wide snapshot. As far as we can tell, Archidekt’s curated lists overlap heavily with EDHPowerLevel’s because both trace back to the official Commander Rules Committee designation, but we did not audit the two for exact equality. This shared lineage is part of the circularity caveat in Section 8.1.

- `game_changer_count`: count of cards with `oracleCard.gameChanger = true`.
- `extra_turns_count`: count of cards with `oracleCard.extraTurns = true`.
- `tutor_count`: count of cards with `oracleCard.tutor = true`.
- `mass_land_denial_count`: count of cards with `oracleCard.massLandDenial = true`.
- `two_card_combo_singleton_count`: count of cards flagged as a singleton piece of a known two-card combo by Archidekt’s combo registry. Empirically zero across the entire collected dataset (the underlying `twoCardComboSingleton` flag is never set on the cards Archidekt returns for our deck pool), so removed by the zero-variance filter.

C.7 Combo density

Combo membership is read from Archidekt’s combo registry embedded in the deck payload at collection time (May 2026). We do not re-derive combos against the Commander Spellbook database; we use whatever combo references Archidekt happens to attach to each card. Like the bracket flags above, this is a frozen snapshot for the whole dataset.

- `cards_with_atomic_combos_count`, `cards_with_potential_combos_count`: number of mainboard cards that participate, respectively, in at least one atomic combo or at least one potential combo according to Archidekt’s combo data.
- `atomic_combo_refs_total`, `potential_combo_refs_total`: total references summed across cards (a card that participates in three combos contributes 3).
- `unique_atomic_combo_refs_count`, `unique_potential_combo_refs_count`: number of distinct combo identifiers referenced by the deck.
- `two_card_combo_ids_total`: total references across cards to two-card combos.
- `unique_two_card_combo_ids_count`: number of distinct two-card combo identifiers referenced by the deck.

C.8 Popularity, salt, and price

All three families drift over time, and we use a single collection-time snapshot (May 2026) for the whole dataset rather than per-deck values from each deck’s creation date. Cross-validation does not fix this temporal mismatch; see Section 8.1.

- `edhrec_rank_mean`, `edhrec_rank_median`, `edhrec_rank_std`: aggregate statistics of EDHREC rank across mainboard cards. Cards with missing EDHREC rank are excluded from these statistics; the resulting missing value is imputed downstream by the fold-fitted median (Appendix B).
- `salt_mean`, `salt_median`, `salt_std`: aggregate statistics of EDHREC salt score, treated analogously.
- `price_total`, `price_mean`, `price_median`, `price_std`: aggregate statistics of paper non-foil price in USD. Vendor preference is TCGPlayer market, falling through Card Kingdom, CardMarket, Star City Games, MagicPlayer, and CardTrader. `price_total` is winsorized at the training-fold 99th percentile before modeling.

C.9 Rarity

- `common_count`, `uncommon_count`, `rare_count`, `mythic_count`: counts of mainboard cards by printing-level rarity, taken from the specific printing the deck uses.

C.10 Keywords

Keywords are extracted from `oracleCard.keywords`.

- `keyword_total_count`: total keyword occurrences across the mainboard (a card with two keywords contributes 2).
- `distinct_keyword_count`: number of distinct keyword strings observed in the mainboard.
- `flying_count`, `trample_count`, `haste_count`, `hexproof_count`, `ward_count`, `indestructible_count`, `lifelink_count`, `menace_count`, `vigilance_count`, `deathtouch_count`, `flash_count`, `equip_count`, `annihilator_count`, `cascade_count`: per-keyword card counts for the tracked evergreen and high-impact keywords.
- `protection_keyword_count`: count of cards whose keyword list contains any `Protection from ...` variant.

C.11 Super- and subtypes

- `legendary_count`, `snow_count`: counts of cards with the corresponding supertype.
- `distinct_subtype_count`: number of distinct subtypes observed across the mainboard.
- `most_common_subtype_count`: total count of cards belonging to the most frequent subtype in the deck (a tribal proxy).
- `equipment_subtype_count`, `aura_subtype_count`, `vehicle_subtype_count`: counts of cards with the corresponding subtype.

C.12 Multiface layouts

- `multiface_card_count`: count of cards that expose more than one face.
- `layout_normal_count`, `layout_transform_count`, `layout_modal_dfc_count`, `layout_split_count`, `layout_adventure_count`: per-layout card counts for the tracked layouts.
- `cards_with_faces_count`: count of cards whose `faces` array is non-empty.
- `total_face_count`: total number of faces summed across the deck.
- `max_faces_on_card`: maximum face count observed on any single card.

D Spot-Checking — Full Results

This appendix carries the full spot-checking results, the software versions used, and the default hyperparameters referenced in Section 4.

Software versions. Python 3.11, scikit-learn 1.5, scipy 1.13, numpy 1.26, pandas 2.2. Hold-out splits use `StratifiedShuffleSplit` (80/20, 5 seeds) and are reproducible from the seed list $\{1, 2, 3, 4, 5\}$.

Default hyperparameters. Each candidate runs with its scikit-learn 1.5 defaults, except where the default is not viable on this scale. The notable defaults are:

- `DecisionTreeClassifier`: `criterion="gini"`, no depth limit.
- `RandomForestClassifier`: 100 trees, `max_features="sqrt"`, `n_jobs=-1`.
- `HistGradientBoostingClassifier`: `max_iter=100`, `learning_rate=0.1`, `max_leaf_nodes=31`.
- `MultinomialNB`: `alpha=1.0`, BC representation.
- `GaussianNB`: `var_smoothing=1e-9`, DF representation.
- `LogisticRegression`: `C=1.0`, `penalty="l2"`, `max_iter=1000`; solver `lbfgs` on DF and `saga` on BC.
- `LinearSVC`: `C=1.0`, `penalty="l2"`, `loss="squared_hinge"`, `max_iter=5000`.
- `KNeighborsClassifier`: `n_neighbors=5`, `weights="uniform"`.

`StandardScaler` is applied to DF for the scale-sensitive algorithms; tree-based models and `GaussianNB` receive unscaled features.

BC vocabulary threshold. `bc_min_df` was selected by sweeping $\{5, 10, 20\}$ on the BC representation. `min_df = 10` produced the highest mean macro-F1 across seeds and is fixed for all subsequent spot-checking and nested CV runs.

Per-algorithm macro-F1. Table 11 reports mean (and standard deviation) of macro-F1 across the five hold-out seeds. The five highest-ranked algorithms per representation are kept; their union A_{union} defines the algorithm set carried forward to nested CV.

Table 11: Spot-checking macro-F1 (mean $\pm$ std over 5 stratified 80/20 hold-outs) per algorithm and representation, with the top-5-per-representation selection. ✓ marks membership in A_{union} .

Algorithm	BC mean	BC std	DF mean	DF std	In A_{union}
Gradient Boosting	0.6342	0.0064	0.6850	0.0049	✓
Logistic Regression	0.5927	0.0066	0.6672	0.0050	✓
Random Forest	0.5243	0.0119	0.6686	0.0103	✓
Linear SVC	0.5459	0.0107	0.6225	0.0112	✓
Decision Tree	0.5445	0.0041	0.6042	0.0102	✓
Naive Bayes	0.5570	0.0070	0.5792	0.0080	✓
KNN	0.4541	0.0177	0.5241	0.0113	—

E Training Verification and Statistical Tests

Before any voting or model selection happens, the pipeline runs a verification phase as a sanity gate. It checks that every expected model is present, every outer fold ran to completion, and every model used the same fold identities. Only after those checks pass do we report paired comparisons.

To check whether models are memorizing commander-specific patterns, a 5-fold `GroupKFold` grouped by commander identity is also run. Table 13 shows the two models we later use for interpretation. The

Table 12: Model verification checks.

Check	Result
Expected individual models	12/12 present
Outer folds per model	15/15 for every model
Shared fold identities	Same repeat/fold IDs across all models
Inner-grid size	24 configurations per algorithm/fold
Best hyperparameters	Rank-1 inner macro-F1 recovered from CV results
Training target	y_{arch} only; y_{calc} excluded from features

Table 13: Commander-grouped robustness check for selected models.

Model	Nested CV macro-F1	GroupKFold macro-F1	Group - nested
df_gradient_boosting	0.6908	0.7021	+0.0113
bc_gradient_boosting	0.6433	0.6496	+0.0063

grouped scores stay close to the stratified nested-CV scores, which is consistent with the top models not depending only on the commander.

All paired statistical tests use the same 15 outer scores for every model. The Friedman, Nemenyi, and Wilcoxon outcomes themselves are summarized in Section 5.5.

F Voting and Model Selection Details

Voting reads saved OOF predictions and never retrains a base model. Member lists are fixed by the global macro-F1 ranking over the individual models; hard-vote ties are broken by the summed macro-F1 of the members voting for each class, with the smaller bracket as a deterministic fallback. The full member lists are in Table 14; interpretability is performed on the best individual DF and BC models (Section 5.7) rather than on the ensemble.

Table 14: Voting ensemble members.

Ensemble	Members
voting_top3	DF Gradient Boosting; DF Random Forest; DF Decision Tree
voting_top5	voting_top3 + DF Logistic Regression + DF Linear SVC
voting_top7	voting_top5 + BC Gradient Boosting + BC Random Forest